

Module 3 — ML Algorithms from Scratch

2 coding questions on the real test, ~20-25 min each, ~15-25 lines of code.

Rules (same as real test):

- No sklearn, pytorch, tensorflow, keras
- `numpy`, `math`, `collections` are fine
- Only fill in the `#` implement this sections

Algorithms to practice: kNN, k-Means, Decision Trees, GMMs, Matrix Normalization, Bagging, Forward Propagation

1 — k-Nearest Neighbors

Skeleton matches the official CodeSignal PDF exactly.

Data format:

- `train_data` : list of rows like `[f1, f2, f3, ..., label]` — last element is the class label (a float)
- `test_data` : list of rows like `[f1, f2, f3, ...]` — no label
- `k` : int, number of neighbors to use

What you implement:

- `euc_dist(value1, value2)` : takes two lists of floats of the same length, returns a single float (the Euclidean distance)
- `k_neighbors(train_data, test_case, k)` : takes the full training set, one test row (no label), and `k`; returns a list of `k` rows from `train_data` (the nearest ones, each including their label as the last element)

```
In [1]: import numpy as np
```

```
In [2]: import collections
```

```
In [21]: np.array(train_data)[:,:-1]
```

```
Out[21]: array([[ -2.6,  1.9,  2. ,  1. ],
                [ -2.8,  1.7, -1.2,  1.5],
                [  2. , -0.9,  0.3,  2.3],
                [ -1.5, -0.1, -1.6, -1.1],
                [ -1. , -0.6, -1.2, -0.7],
                [ -0.3,  1.2,  2.6,  0.2],
                [ -1.8, -1.3, -0.1, -1.2],
                [  0.2,  1.2, -0.6, -1.3],
                [ -5.2,  0.3,  0.2,  2.2],
                [ -0.8, -0.1,  1.5, -0.1],
                [ -2.3,  0.3,  0.8,  0.7],
                [  0.2,  3. ,  3.6, -0.9],
                [  1.7, -0.8, -0. ,  2. ],
                [  2.8,  0.8,  1.8, -0.7]])
```

```
In [23]: import math

def euc_dist(value1, value2):
    # value1, value2: lists of floats, same length
    # return: float
    # implement this
    value1, value2 = (np.array(value1), np.array(value2))
    return np.linalg.norm(value1-value2)

def k_neighbors(train_data, test_case, k):
    # train_data: list of rows, each row is [f1, f2, ..., label]
    # test_case: one row [f1, f2, ...] with NO label
    # k: int
    # return: list of k rows from train_data (each still has its label as la
    # implement this
    train_data, test_case = (
        np.array(train_data), np.array(test_case))
    distances = np.array(
        [ euc_dist(train_vec[:-1], test_case)
          for train_vec in train_data ])
    rank_ids = np.argsort(distances)
    closest_k_ids = rank_ids[:k]
    return train_data[closest_k_ids]

# --- pre-written, do not modify ---
def get_label(train_data, test_case, k):
    neighbors = k_neighbors(train_data, test_case, k)
    labels = [row[-1] for row in neighbors]
    return max(set(labels), key=labels.count)

def solution(train_data, test_data, k):
    return [get_label(train_data, row, k) for row in test_data]

# -----

train_data = [
    [-2.6, 1.9, 2.0, 1.0, 1.0], [-2.8, 1.7, -1.2, 1.5, 2.0],
    [2.0, -0.9, 0.3, 2.3, 0.0], [-1.5, -0.1, -1.6, -1.1, 0.0],
    [-1.0, -0.6, -1.2, -0.7, 0.0], [-0.3, 1.2, 2.6, 0.2, 1.0],
    [-1.8, -1.3, -0.1, -1.2, 0.0], [0.2, 1.2, -0.6, -1.3, 1.0],
    [-5.2, 0.3, 0.2, 2.2, 2.0], [-0.8, -0.1, 1.5, -0.1, 0.0],
    [-2.3, 0.3, 0.8, 0.7, 2.0], [0.2, 3.0, 3.6, -0.9, 1.0],
```

```

    [1.7, -0.8, -0.0, 2.0, 0.0], [2.8, 0.8, 1.8, -0.7, 2.0]
]
test_data = [
    [-0.1, 1.4, 0.4, -1.0], [-1.3, 0.2, -1.3, -0.8],
    [-1.1, 1.5, -2.3, -2.5], [0.2, 2.0, -0.1, -0.8],
    [-0.3, -1.6, -3.4, -1.4]
]
assert solution(train_data, test_data, 3) == [1.0, 0.0, 0.0, 1.0, 0.0]
print("kNN passed")

```

kNN passed

► Solution

```

def euc_dist(value1, value2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(value1,
value2)))

def k_neighbors(train_data, test_case, k):
    # compute distance from test_case to each training row
    (excluding label)
    distances = [(row, euc_dist(test_case, row[:-1])) for row in
train_data]
    distances.sort(key=lambda x: x[1])
    return [row for row, _ in distances[:k]]

```

2 — k-Means Clustering

Data format:

- `data` : list of points, each point is a list of floats e.g. `[[1.0, 2.0], [3.0, 4.0], ...]`
- `centroids` : list of points (same format), one per cluster
- `k` : int, number of clusters (= `len(centroids)`)
- `assignments` : list of ints, one per data point — the index of the cluster it belongs to

What you implement:

- `euc_dist(a, b)` : same as before — two lists of floats → float
- `assign_clusters(data, centroids)` : for each point, find the nearest centroid → return list of ints (cluster indices), length = `len(data)`
- `update_centroids(data, assignments, k)` : for each cluster, compute mean of its assigned points → return list of `k` centroids (each a list of floats)

```

In [31]: f=[0.]*10
         f.append(3)

```

```

In [32]: f

```

Out[32]: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 3]

```
In [33]: import math

def euc_dist(a, b):
    # a, b: lists of floats, same length
    # return: float
    # implement this
    a, b = np.array(a), np.array(b)
    return np.linalg.norm(a-b)

def assign_clusters(data, centroids):
    # data: list of points [[f1,f2,...], ...]
    # centroids: list of points [[f1,f2,...], ...]
    # return: list of ints, length = len(data)
    # result[i] = index of nearest centroid to data[i]
    # implement this
    #
    result = [0]*len(data)
    for j, data_j in enumerate(data):
        result[j] = np.argmin(
            [euc_dist(data_j, centroid_k) \
             for centroid_k in centroids]
        )

    return result

def update_centroids(data, assignments, k):
    # data: list of points
    # assignments: list of ints (cluster index for each point)
    # k: number of clusters
    # return: list of k new centroids (each a list of floats = mean of assign)
    # implement this
    assignments = np.array(assignments)
    data = np.array(data)
    result = []
    for i in range(k):
        result.append(
            (np.mean(
                data[assignments == i], axis=0)).tolist() )

    return result

# --- pre-written, do not modify ---
def solution(data, initial_centroids, max_iter):
    centroids = initial_centroids
    assignments = None
    for _ in range(max_iter):
        assignments = assign_clusters(data, centroids)
        centroids = update_centroids(
            data, assignments, len(centroids))
    return assignments

# -----

data = [[1, 1], [1, 2], [2, 1], [8, 8], [8, 9], [9, 8]]
```

```

result = solution(data, [[1, 1], [9, 9]], max_iter=10)
assert result[:3] == [result[0]] * 3
assert result[3:] == [result[3]] * 3
assert result[0] != result[3]
print("k-Means passed")

```

k-Means passed

► Solution

```

def euc_dist(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def assign_clusters(data, centroids):
    assignments = []
    for point in data:
        dists = [euc_dist(point, c) for c in centroids]
        assignments.append(dists.index(min(dists)))
    return assignments

def update_centroids(data, assignments, k):
    new_centroids = []
    for i in range(k):
        pts = [data[j] for j in range(len(data)) if assignments[j] == i]
        if pts:
            centroid = [sum(p[d] for p in pts) / len(pts) for d in range(len(pts[0]))]
        else:
            centroid = [0.0] * len(data[0])
        new_centroids.append(centroid)
    return new_centroids

```

3 — Decision Tree (single best split)

Data format:

- `train_data` : list of feature vectors e.g. `[[1.0, 2.0], [3.0, 4.0], ...]`
- `train_labels` : list of ints (class labels), same length as `train_data` — `train_labels[i]` is the label for `train_data[i]`
- `test_data` : list of feature vectors, no labels
- `labels` : a list of ints (used internally in gini)

What you implement:

- `gini(labels)` : takes a list of ints → returns float. Gini impurity = $1 - \sum(p_c^2)$ where p_c = fraction of labels equal to class c
- `best_split(data, labels)` : tries every feature and every midpoint threshold; returns `(feature_index, threshold)` as a tuple — the split that gives the

lowest weighted Gini across the two resulting groups

```
In [37]: np.sum(np.array([3,4,5]) == 1)
```

```
Out[37]: 0
```

```
In [52]: a=set([4,4,5,9,9,9,1,2,3])
```

```
b=list(a)
```

```
In [54]: b.sort()
```

```
In [75]: b=np.array(data)[: ,0]
```

```
In [80]: np.array(list(a))
```

```
Out[80]: array([1, 2, 3, 4, 5, 9])
```

```
In [92]: b.tolist()
```

```
Out[92]: [1, 1, 2, 8, 8, 9]
```

```
In [93]: b
```

```
Out[93]: array([1, 1, 2, 8, 8, 9])
```

```
In [89]: type(b) == np.ndarray
```

```
Out[89]: True
```

```
In [95]: def gini(labels):
    # labels: list of ints e.g. [0, 0, 1, 1, 0]
    # return: float in [0, 0.5] - 0 means pure, 0.5 means maximally mixed (b
    # formula: 1 - sum over each class c of (count(c)/n)^2
    # implement this
    if type(labels) == np.ndarray:
        labels=labels.tolist()
    n_labels=len(labels)
    g = 1.
    if n_labels == 0:
        return g

    for l in set(labels):
        g -= (labels.count(l) / n_labels )**2

    return g

def best_split(data, labels):
    # data: list of feature vectors [[f1,f2,...], ...]
    # labels: list of ints, same length
    # return: (best_feature_index: int, best_threshold: float)
    # strategy: for each feature,
    #     try thresholds = midpoints between consecutive sorted unique values
```

```

#         split = left (<=thresh) and right (>thresh)
#         score = weighted average gini of left and right
#         return the (feature, threshold) with lowest score
# implement this
nfeats = len(data[0])
data=np.array(data)
nlabels=len(labels)
labels=np.array(labels)
best_threshold_per_feature = np.zeros(nfeats)
best_score_per_feature = np.zeros(nfeats)

for n in range(nfeats):
    one_feat_all_data = data[:,n]
    ofad = one_feat_all_data
    one_feat_all_data_sorted = np.sort(np.array(list(set(one_feat_all_da
    ofads=one_feat_all_data_sorted
    midpoints = [(ofads[i]+ofads[i+1])/2 for i in range(len(ofads)-1)]
    scores = []
    for th in midpoints:
        left_labels =labels[ofad<=th]
        right_labels=labels[ofad>th]
        score = (
            len(left_labels)/nlabels)*gini(left_labels)
        + (len(right_labels)/nlabels)*gini(right_labels)
        scores.append(score)

    best_score_idx = np.argmin(np.array(scores))
    best_threshold_per_feature[n] = midpoints[best_score_idx]
    best_score_per_feature[n] = scores[best_score_idx]

lowest_score_idx=np.argmin(best_score_per_feature)
return (lowest_score_idx, best_threshold_per_feature[lowest_score_idx])

# --- pre-written, do not modify ---
def majority_label(labels):
    return max(set(labels), key=labels.count)

def solution(train_data, train_labels, test_data):
    feat_idx, threshold = best_split(train_data, train_labels)
    left_labels = [train_labels[i] for i, row in enumerate(train_data) \
                    if row[feat_idx] <= threshold]
    right_labels = [train_labels[i] for i, row in enumerate(train_data) \
                    if row[feat_idx] > threshold]
    left_pred = majority_label(left_labels) if left_labels else 0
    right_pred = majority_label(right_labels) if right_labels else 0
    return [left_pred if row[feat_idx] <= threshold
            else right_pred for row in test_data]

# -----
train_data = [[1, 2], [2, 1], [8, 9], [9, 8]]
train_labels = [0, 0, 1, 1]
test_data = [[1, 1], [9, 9]]
assert solution(train_data, train_labels, test_data) == [0, 1]
print("Decision tree passed")

```

Decision tree passed

► Solution

```
def gini(labels):
    if not labels:
        return 0
    n = len(labels)
    return 1 - sum((labels.count(c) / n) ** 2 for c in set(labels))

def best_split(data, labels):
    best_feat, best_thresh, best_score = None, None, float('inf')
    n = len(data)
    for feat in range(len(data[0])):
        values = sorted(set(row[feat] for row in data))
        thresholds = [(values[i] + values[i+1]) / 2 for i in
range(len(values) - 1)]
        for thresh in thresholds:
            left = [labels[i] for i, row in enumerate(data) if
row[feat] <= thresh]
            right = [labels[i] for i, row in enumerate(data) if
row[feat] > thresh]
            score = (len(left) / n) * gini(left) + (len(right) / n)
* gini(right)
            if score < best_score:
                best_score, best_feat, best_thresh = score, feat,
thresh
    return best_feat, best_thresh
```

4 — Matrix Normalization

Data format:

- `matrix` : list of rows, each row is a list of floats e.g. `[[2, 4], [4, 8], [6, 12]]`
- `col` : int, column index
- `mean` : float, precomputed column mean (passed into `col_std` so you don't recompute it)

What you implement:

- `col_mean(matrix, col)` : return the mean (float) of column `col` across all rows
- `col_std(matrix, col, mean)` : return the population standard deviation (float) of column `col` — use the given `mean`, don't recompute it
- `solution(matrix)` : return a new matrix (list of lists) where each column is z-score normalized: $(x - \text{mean}) / \text{std}$

In []:

```

In [112]: def col_mean(matrix, col):
# matrix: list of rows (list of lists of floats)
# col: int index
# return: float – mean of matrix[0][col], matrix[1][col], ...
# implement this
return np.mean(np.array(matrix)[: ,col])

def col_std(matrix, col, mean):
# matrix: list of rows
# col: int index
# mean: float (already computed – use it, don't recompute)
# return: float – population std = sqrt(mean of squared deviations)
# implement this
return np.sqrt(np.mean((np.array(matrix)[: ,col] - mean)**2))

def solution(matrix):
# matrix: list of rows (list of lists of floats)
# return: new matrix
# (same shape, list of lists) where each column has mean=0, std=1
# implement this
ncols = len(matrix[0])
nrows = len(matrix)
for i in range(ncols):
    cm = col_mean(matrix, i)
    cstd = col_std(matrix, i, cm)
    for j in range(nrows):
        matrix[j][i] -= cm
        if cstd != 0:
            matrix[j][i] /= cstd
        else:
            matrix[j][i] = 0
    return matrix

matrix = [[2, 4], [4, 8], [6, 12]]
result = solution(matrix)
for col in range(2):
    col_vals = [result[row][col] for row in range(len(result))]
    assert abs(sum(col_vals) / len(col_vals)) < 1e-9, "mean not 0"
print("Normalization passed")

```

Normalization passed

► Solution

```

def col_mean(matrix, col):
    return sum(row[col] for row in matrix) / len(matrix)

def col_std(matrix, col, mean):
    variance = sum((row[col] - mean) ** 2 for row in matrix) / len(matrix)
    return variance ** 0.5

def solution(matrix):

```

```

n_cols = len(matrix[0])
means = [col_mean(matrix, j) for j in range(n_cols)]
stds = [col_std(matrix, j, means[j]) for j in range(n_cols)]
return [[(row[j] - means[j]) / stds[j] if stds[j] != 0 else 0
          for j in range(n_cols)] for row in matrix]

```

5 — Forward Propagation

Data format:

- `network` : list of layers. Each layer is a list of neurons. Each neuron is a dict: `{'weights': [w1, w2, ..., bias]}` — the **last weight is the bias**.
- `inputs` : list of floats — the input values to the network
- A neuron with `n` inputs has `n+1` weights: `weights[0..n-1]` are the input weights, `weights[n]` is the bias

What you implement:

- `activate_neuron(weights, inputs)` : takes `weights` (list of floats, last is bias) and `inputs` (list of floats); returns float: `dot(weights[:-1], inputs) + weights[-1]`
- `forward_propagate(network, inputs)` : pass inputs through every layer; hidden layers use ReLU, output layer uses sigmoid; store result in `neuron['output']` ; return list of outputs from the final layer

In [117... `f = relu`

In [118... `f(2)`

Out[118... 2

```

In [116... import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def relu(x):
    return max(0, x)

```

```

In [120... def activate_neuron(weights, inputs):
    # weights: list of floats, length = len(inputs) + 1, last element is bias
    # inputs: list of floats
    # return: float = sum(weights[i] * inputs[i]) + weights[-1]
    # implement this
    weights=np.array(weights)
    inputs=np.array(inputs)
    return np.dot(weights[:-1], inputs) + weights[-1]

```

```

def forward_propagate(network, inputs):
    # network: list of layers (see format above)
    # inputs: list of floats (the initial input values)
    # for each layer:
    #     compute activate_neuron for each neuron using current inputs
    #     apply relu if hidden layer, sigmoid if last layer
    #     store result in neuron['output']
    #     outputs of this layer become inputs to the next
    # return: list of floats – outputs of the final layer
    # implement this
    nlayers = len(network)
    # inputs_curr = inputs.copy()
    for i, layer in enumerate(network):
        outputs = [0.] * len(layer)
        f = sigmoid if i == nlayers-1 else relu
        for j, neuron in enumerate(layer):
            outputs[j] = f(
                activate_neuron(
                    neuron["weights"], inputs))
            neuron["output"] = outputs[j]
        inputs = outputs.copy()
    return inputs

# network: 2 inputs → 2 hidden neurons (relu) → 1 output (sigmoid)
# each neuron's weights list = [w_for_input1, w_for_input2, bias]
network = [
    [{'weights': [0.5, 0.5, 0.0]}, {'weights': [-0.5, 0.5, 0.0]}], # hidden
    [{'weights': [1.0, 1.0, -0.5]}] # output
]
inputs = [1.0, 0.0]
outputs = forward_propagate(network, inputs)
assert len(outputs) == 1
assert 0 < outputs[0] < 1
print(f"Forward prop output: {outputs[0]:.4f} – passed")

```

Forward prop output: 0.5000 – passed

► Solution

```

def activate_neuron(weights, inputs):
    return sum(w * x for w, x in zip(weights[:-1], inputs)) +
    weights[-1]

def forward_propagate(network, inputs):
    current = inputs
    for i, layer in enumerate(network):
        is_last = (i == len(network) - 1)
        new = []
        for neuron in layer:
            z = activate_neuron(neuron['weights'], current)
            neuron['output'] = sigmoid(z) if is_last else relu(z)
            new.append(neuron['output'])
        current = new
    return current

```

Problem 6 — Bagging

Implement two helper functions used in a bagging classifier.

`predict_1nn(train_sample, test_point)`

- `train_sample` : list of (features, label) pairs — e.g. `[[[1.0, 2.0], 0], ([3.0, 4.0], 1)]`
- `test_point` : one feature vector — e.g. `[1.5, 2.5]`
- Find the training point closest to `test_point` (Euclidean distance)
- Return that point's label

`bootstrap_sample(data, labels, seed)`

- `data` : list of feature vectors; `labels` : list of ints, same length
- Set `random.seed(seed)`, then draw `len(data)` indices **with replacement** from `range(len(data))`
- Return two lists: the feature vectors at those indices, and the labels at those indices
- Some indices will repeat, some will be missing — that's intentional

```
In [8]: import math
import random
import numpy as np

def euc_dist(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def predict_1nn(train_sample, test_point):
    # train_sample: list of (features, label) tuples
    # e.g. [[1.0, 2.0], 0), ([3.0, 4.0], 1), ...]
    # test_point: list of floats e.g. [1.5, 2.5]
    # return: int — label of the single nearest neighbor
    # implement this
    closest_idx = np.argmin(
        np.array([euc_dist(
            x[0], test_point) for x in train_sample]) )
    return train_sample[closest_idx][1]

def bootstrap_sample(data, labels, seed):
    # data: list of feature vectors
    # labels: list of ints
    # seed: int (set random.seed(seed) at the start)
    # return: (sample_data, sample_labels) — two lists of length len(data)
    # sampled with replacement (some rows may repeat, some may be ab
    # implement this
    pass
    random.seed(seed)
    indcs = np.array(
```

```

        [ random.randint(0, len(data)-1) for i in range(len(data))]
    return (np.array(data)[inds].tolist(),
            np.array(labels)[inds].tolist())

# --- pre-written, do not modify ---
def solution(train_data, train_labels, test_data, n_estimators, seed=42):
    all_predictions = []
    for i in range(n_estimators):
        sample_data, sample_labels = bootstrap_sample(
            train_data, train_labels, seed=i)
        preds = [predict_1nn(
            list(zip(sample_data, sample_labels)), pt) for pt in test_data]
        all_predictions.append(preds)
    final = []
    for j in range(len(test_data)):
        votes = [all_predictions[i][j] for i in range(n_estimators)]
        final.append(max(set(votes), key=votes.count))
    return final

# -----

train_data = [[1, 1], [1, 2], [2, 1], [8, 8], [8, 9], [9, 8]]
train_labels = [0, 0, 0, 1, 1, 1]
test_data = [[1, 1], [9, 9]]
assert solution(train_data, train_labels, test_data, n_estimators=10) == [0,
print("Bagging passed")

```

Bagging passed

► Solution

```

def predict_1nn(train_sample, test_point):
    best_dist, best_label = float('inf'), None
    for features, label in train_sample:
        d = euc_dist(features, test_point)
        if d < best_dist:
            best_dist, best_label = d, label
    return best_label

def bootstrap_sample(data, labels, seed):
    random.seed(seed)
    n = len(data)
    indices = [random.randint(0, n - 1) for _ in range(n)]
    return [data[i] for i in indices], [labels[i] for i in indices]

```

7 — Gaussian Mixture Models (E-step)

The E-step computes how much each Gaussian component "claims" each data point.

Data format:

- data : list of floats e.g. [1.0, 5.0, 9.0] — 1D data points

- `mus` : list of floats — mean of each component e.g. `[1.0, 9.0]`
- `sigmas` : list of floats — std dev of each component e.g. `[1.0, 1.0]`
- `pis` : list of floats — mixing weights, sum to 1 e.g. `[0.5, 0.5]`

What you implement:

- `gaussian_pdf(x, mu, sigma)` : takes three floats, returns float — the value of the 1D Gaussian PDF at `x`
- `e_step(data, mus, sigmas, pis)` : for each data point, compute the responsibility of each component; return a 2D list where `result[i][k]` = responsibility of component `k` for point `i` (each row sums to 1)

Formula: $r[i][k] = \text{pis}[k] * N(\text{data}[i] \mid \text{mus}[k], \text{sigmas}[k]) / \sum_j \text{pis}[j] * N(\text{data}[i] \mid \text{mus}[j], \text{sigmas}[j])$

```
In [10]: a=[3,4,5]
```

```
In [11]:
```

TypeError

Traceback (most recent call last)

Cell In[11], line 1

----> 1 a/4

TypeError: unsupported operand type(s) for /: 'list' and 'int'

```
In [13]: import math
```

```
def gaussian_pdf(x, mu, sigma):
    # x: float - data point
    # mu: float - mean
    # sigma: float - standard deviation
    # return: float - (1 / (sigma * sqrt(2*pi))) * exp(-0.5 * ((x - mu) / sigma)**2)
    # implement this
    return (1 / (sigma * np.sqrt(2*np.pi))) * np.exp(-0.5 * ((x-mu)/sigma)**2)

def e_step(data, mus, sigmas, pis):
    # data: list of floats, length n
    # mus, sigmas, pis: lists of floats, length k (one per component)
    # return: list of n lists, each of length k
    #         result[i][k] = responsibility of component k for point data[i]
    #         each result[i] must sum to 1.0
    # implement this
    n = len(data)
    n_comps = len(mus)
    result = [0.]*n
    for i in range(n):
        result[i] = [pis[k]*gaussian_pdf(
            data[i], mus[k], sigmas[k]) for k in range(n_comps)]
        np_result=np.array(result[i])
        result[i] = (np_result / np_result.sum(keepdims=True)).tolist()
    return result
```

```

data    = [1.0, 5.0, 9.0]
mus     = [1.0, 9.0]
sigmas  = [1.0, 1.0]
pis     = [0.5, 0.5]

r = e_step(data, mus, sigmas, pis)
assert r[0][0] > 0.99, \
    f"point 1.0 should be almost all component 0, got {r[0][0]:.4f}"
assert r[2][1] > 0.99, \
    f"point 9.0 should be almost all component 1, got {r[2][1]:.4f}"
for i in range(len(data)):
    assert abs(sum(r[i]) - 1.0) < 1e-9, f"row {i} doesn't sum to 1"
print("GMM E-step passed")

```

GMM E-step passed

► Solution

```

def gaussian_pdf(x, mu, sigma):
    coeff = 1 / (sigma * math.sqrt(2 * math.pi))
    return coeff * math.exp(-0.5 * ((x - mu) / sigma) ** 2)

def e_step(data, mus, sigmas, pis):
    responsibilities = []
    for x in data:
        raw = [pis[k] * gaussian_pdf(x, mus[k], sigmas[k]) for k in
range(len(mus))]
        total = sum(raw)
        responsibilities.append([r / total for r in raw])
    return responsibilities

```